

阅读文献笔记

HW1 挑战内容 6 · 参考: SGLang/RadixAttention (NeurIPS 2024)、vLLM/PagedAttention (SOSP 2023)、Mooncake (FAST'25)

1. Efficient Memory Management for LLM Serving with PagedAttention (vLLM, SOSP 2023)

Kwon et al. — <https://dl.acm.org/doi/10.1145/3600006.3613165>

- **观察:** LLM 服务吞吐瓶颈常不在计算而在 KV cache 显存管理: 预分配连续空间带来内部碎片 (按最大长度预留) 与外部碎片, 实测仅 20%–38% 的 KV 显存存了有效数据。
- **方法:** 借鉴虚拟内存分页: KV cache 切成固定大小 block (如 16 token/块), block table 做逻辑→物理映射; 注意力内核按 block 表收集 K/V。同一 prompt 的多个采样序列共享物理 block (copy-on-write)。
- **结果:** 相比 FasterTransformer、Orca, 吞吐提升 2–4×; 长序列下收益更大。
- **我的评论:** 它解决的是"物理层"问题——显存碎屑; 前缀共享只是 block 级的附带能力, 匹配靠 block 哈希链, 没有树结构的分支语义。
- **与作业的联系:** SGLang 的 `token_to_kv_pool_allocator` 同样按页 (page_size) 分配, HW2 实验里 radix 插入后释放"未对齐尾部"正是页对齐的体现。

2. SGLang / RadixAttention (NeurIPS 2024)

Zheng et al. — RadixAttention: 前缀 KV 的自动复用 + SGLang 前端受限生成语言

- **问题:** 真实负载中大量重复前缀 (few-shot、多轮、agentic 循环), 但系统间 KV 不共享导致重复 Prefill。
- **方法:** ① 运行时: 所有请求的 KV cache 挂到一棵 radix tree 上, 边即 token 序列、节点即 KV 段; 最长前缀匹配 $O(L)$; 节点引用计数保护使用中分支, LRU 在分支粒度驱逐; ② 前端: 一套可组合的受限生成语言 (跳跃、分叉、正则/JSON 约束), 配合缓存使多步程序共享上下文。
- **结果:** 在多轮对话、few-shot、beam search、tree-of-thought 等负载上, 吞吐最高约 5× 于当时的 vLLM/TGI 基线。
- **我的评论:** 关键洞察是"前缀结构是负载的内生属性", 与其让用户显式声明缓存, 不如让系统自动发现; 代价是树维护与驱逐策略的复杂度。
- **与作业的联系:** HW2 的共享前缀组命中率 89.8%、Prefill token -89.7%, 正是论文机制在 v0.5.14 上的直接可测效果; 而 TPOT 不变 (-1.1%) 也印证 Decode 收益有限。

3. Mooncake: A KVCache-Centric Disaggregated Architecture (FAST'25) 及其 trace

Qin et al., Moonshot AI — <https://github.com/kvcache-ai/Mooncake> (FAST25-release)

- **思想:** 把 Prefill 与 Decode 解耦到不同机器池, 中间以 KVCache 为中心做传输与调度; 前缀缓存 (含跨机/跨层存储) 是全局调度的第一等公民。
- **trace:** 公开的生产级请求级 trace (timestamp、input_length、output_length、hash_ids)。hash_ids 本身就编码了前缀块的重用结构——本作业 HW1 用其中的 arxiv-trace 采样 24 条 真实记录构造 workload, HW2 的对照实验思想 (可复用前缀 vs 不可复用前缀) 也与之一脉相承。
- **启发:** 当缓存从"单机优化"上升为"集群调度中心", 命中率与放置策略直接决定成本; 这也解释了为什么生产 trace 会把 hash_ids 公开——它让任何人都能复现"可复用性"维度的研究。

4. 三个系统一张图 (我的总结)

	PagedAttention (vLLM)	RadixAttention (SGLang)	Mooncake
核心抽象	页式 KV cache + block table	radix tree 组织的 KV 前缀库	KVCache 为中心的 disaggregation
主要目标	消灭显存碎片、提高并发	自动跨请求复用前缀、免重算	集群级 prefill/decode 解耦与全局缓存
共享粒度	block（显式/哈希匹配）	树节点（自动最长前缀）	块哈希（跨实例迁移/复用）
作业中的体现	page_size 对齐、页分配器	命中率 89.8%、Prefill -89.7%	workload 数据来源